

The background of the slide is a photograph of the ITS Rektorat building, a large white structure with a red-tiled roof and a central emblem. It is surrounded by lush green trees. A dark blue rectangular box is overlaid on the lower half of the image, containing the title text in white.

Pengenalan Pemrograman Berbasis Framework Evolusi PHP Native ke Ekosistem Modern & Platform Agentic AI



Pemrograman Berbasis Kerangka Kerja (PBKK)

Pertemuan 1: Pengenalan Pemrograman Berbasis Framework
Evolusi PHP Native ke Ekosistem Modern & Platform Agentic AI

Departemen Teknik Informatika
Fakultas Teknik Elektro dan Informatika Cerdas
Institut Teknologi Sepuluh Nopember Surabaya

Kontrak Perkuliahan & Bobot Evaluasi

Bobot & Aturan SKS

- Kode MK: IF184xxx
- Bobot Kuliah: T=2, P=1
- Teori: 2 SKS (TM)
- Praktikum: 1 SKS (PT+BM)
- Total 16 Pertemuan kelas tatap muka termasuk evaluasi.
- Kehadiran kuliah minimum sesuai dengan peraturan ITS (75%).

Skema Penilaian

- Evaluasi terdiri atas 4 pilar utama dengan bobot seimbang:
 1. Studi Kasus Individu: 25%
 2. UTS Kognitif: 25%
 3. Studi Kasus Kelompok (Proyek Akhir): 25%
 4. UAS Kognitif & Demo: 25%

Referensi Utama

- Matt Stauffer - 'Laravel: Up & Running' (O'Reilly Media, 2nd Edition)
- Dokumentasi Resmi Laravel (<https://laravel.com/docs>)
- Laracasts (<https://laracasts.com>)
- Kode editor: VS Code atau PhpStorm

Visi Proyek Akhir: Platform Agentic AI

Di akhir semester, setiap kelompok beranggotakan 6 mahasiswa akan membangun platform Agentic AI mandiri berbasis Laravel yang dikemas menjadi standalone desktop application menggunakan NativePHP!

Karakteristik Agentic AI App

- UI Reaktif: Antarmuka chat interaktif yang reaktif dibangun menggunakan Laravel Livewire.
- Integrasi LLM: Terhubung langsung ke API LLM lokal (Ollama) gratis atau API Senopati AI ITS.
- Tool Execution: AI mampu memanggil backend task (seperti script, command, job) secara otomatis.

Pilihan Tema Kelompok

Setiap kelompok memilih tema asisten cerdas spesifik:

1. Security Vulnerability Scanner Agent
2. Malware/Virus Scanner Agent
3. Network Port Scanner & Log Analyzer Agent
4. Database Health Checker & Performance Monitor

BAGIAN 1: Dari PHP Native ke Framework

Evolusi PHP, Tantangan Kode Spasial, dan Lahirnya Prinsip DRY

Mengenai Pendekatan PHP Native

Definisi & Konsep Dasar

- PHP Native mengacu pada pembangunan aplikasi web menggunakan bahasa PHP murni tanpa mengadopsi framework eksternal apa pun.
- Seluruh aspek aplikasi ditulis dari nol, mulai dari koneksi database, penanganan request HTTP, autentikasi sesi, penyaringan keamanan, hingga manajemen rute.
- Memberikan kebebasan mutlak bagi developer dalam menstrukturkan file, namun mengalihkan beban pembuatan infrastruktur dasar sepenuhnya ke pundak developer.

Karakteristik PHP Native

- Skala Kecil: Sangat fungsional dan cepat dibuat untuk proyek skala kecil, prototipe instan, atau skrip utilitas sederhana.
- Ketidadaan Aturan Baku: Tidak ada struktur standar. Tiga developer PHP Native berbeda dapat menghasilkan tiga arsitektur direktori proyek yang benar-benar berbeda.
- Rentang Waktu Lama: Menghabiskan waktu lama untuk membangun fitur standar berulang (seperti sistem registrasi, login, & proteksi input) sebelum bisa mulai menulis logika bisnis utama.

Tantangan PHP Native: Spaghetti Code

Spaghetti Code

Merupakan metafora untuk kode program yang tidak terstruktur, melilit, bercampur aduk, dan sangat sulit untuk ditelusuri alurnya. Hal ini terjadi secara alami pada proyek PHP Native berukuran sedang hingga besar.

Mengapa Spaghetti Code Berbahaya?

- Percampuran Kode (Tight Coupling): File PHP tunggal berisi tag HTML untuk antarmuka, query SELECT/INSERT SQL, logika penanganan form, dan kalkulasi bisnis sekaligus.
- Sukar Mengubah Tampilan: Jika desainer ingin merapikan UI, ia berisiko tinggi merusak query database atau logika PHP yang terjalin erat di dalam file HTML.
- Hambatan Kolaborasi: Programmer backend dan desainer frontend harus bekerja pada satu file yang sama, menyebabkan konflik Git yang konstan dan kebingungan tim.
- Tidak Ada Pengujian Otomatis: Kode tidak modular, sehingga unit testing tidak dapat diterapkan.

Kerentanan Keamanan pada PHP Native

Tiga Celah Keamanan Utama Tanpa Proteksi Bawaan

1. SQL Injection (SQLi)

Terjadi ketika input mentah pengguna dimasukkan langsung ke dalam query database string SQL tanpa prepared statements. Penyerang dapat membaca atau menghapus seluruh isi database.

- PHP Native: Developer sering kali lupa menyaring string parameter, atau menggunakan konkatenasi string manual.

2. Cross-Site Scripting (XSS)

Terjadi ketika script JavaScript jahat disisipkan ke input form dan disimpan ke database, lalu dirender mentah-mentah ke halaman web pengguna lain.

- PHP Native: Membutuhkan pembungkusan konstan dengan htmlspecialchars() pada setiap output variabel teks.

3. Cross-Site Request Forgery (CSRF)

Serangan di mana pengguna dipaksa melakukan aksi yang tidak diinginkan pada aplikasi web yang sedang aktif sesinya.

- PHP Native: Tidak ada token perlindungan formulir otomatis. Developer harus merancang mekanisme token sendiri.

Duplikasi Tinggi & Pelanggaran Prinsip DRY

Prinsip DRY (Don't Repeat Yourself)

- DRY adalah prinsip rekayasa perangkat lunak dasar untuk menghindari replikasi logika.
- Setiap logika, kueri database, koneksi, atau komponen tampilan visual hanya boleh dibuat sekali (single source of truth) dan dipanggil ulang saat dibutuhkan.
- Duplikat kode merupakan pintu masuk bagi inkonsistensi sistem dan kesalahan manusia (human error) saat pemeliharaan.

Inkonsistensi pada PHP Native

- Pola include/require manual seperti 'include header.php' atau 'include db.php' sering tersebar di rincian file.
- Ketika struktur menu, koneksi database, atau file layout berubah, developer terpaksa memodifikasi baris kode di puluhan file PHP berbeda.
- Mengakibatkan fragmentasi logika aplikasi di mana perubahan sering kali terlewatkan pada beberapa halaman, menimbulkan bug tak terduga.

Solusi Nyata: Mengapa Menggunakan Framework?

Framework menyediakan kerangka kerja, kumpulan library siap pakai, dan seperangkat aturan/konvensi yang memandu developer membangun aplikasi dengan rapi, aman, dan sangat cepat.

Struktur Baku & Rapi

Memaksa tim mematuhi konvensi yang sama (MVC). Struktur folder rapi, folder app, routes, resources terpisah.

Proteksi Keamanan

Filter otomatis XSS, prepared statements SQLi bawaan, validasi form, CSRF token siap pakai secara default.

Komponen Out-of-the-Box

Tidak perlu mengoding ulang sistem login, email, queue, session, rest api, routing, cache, orm. Semuanya telah terintegrasi.

Model Abstraksi: Menjembatani Masalah

PROBLEM SPACE

(Dunia Nyata)

- Kebutuhan bisnis
- Keinginan klien/pengguna
- Persyaratan sistem
- Tantangan keamanan data
- Logika transaksi yang rumit



ABSTRAKSI

(Framework Laravel)

- MVC Architecture
- ORM (Eloquent Database)
- Routing & Controller
- Middleware & Auth
- Package Ecosystem



SOLUTION SPACE

(Aplikasi Handal)

- Platform teruji & aman
- Kode modular & bersih
- Aplikasi mudah dirawat
- Siap rilis ke produksi
- Kemudahan integrasi AI

BAGIAN 2: Arsitektur Model-View-Controller (MVC)

Konsep Pemisahan Tanggung Jawab (Separation of Concerns) pada Web

Separation of Concerns (SoC)

Mengapa Memisahkan Logika?

- Separation of Concerns (SoC) adalah prinsip desain rekayasa perangkat lunak untuk mengurai program menjadi bagian-bagian terpisah berdasarkan tugasnya.
- Setiap bagian (Concern) harus memiliki tanggung jawab yang spesifik dan terisolasi dari bagian lainnya.
- Mencegah satu komponen merusak komponen lain saat terjadi perubahan.
- Meningkatkan kejelasan struktur kode dan mempermudah pelacakan jika terjadi kesalahan sistem.

MVC sebagai Perwujudan SoC

- Model-View-Controller (MVC) adalah pola arsitektur paling populer untuk implementasi SoC pada aplikasi web.
- Model menangani data dan basis data secara terisolasi.
- View menangani penyajian visual (antarmuka) data kepada pengguna.
- Controller bertindak sebagai konduktor/perekat alur logika yang menghubungkan Model dan View tanpa membiarkan keduanya berinteraksi langsung secara serampangan.

Model: Struktur & Logika Data

Peran & Tanggung Jawab Model

- Representasi Data: Merepresentasikan struktur data riil di dalam aplikasi (misal: entitas Mahasiswa, Dosen, Buku).
- Validasi Logika Data: Memuat aturan validasi tingkat database dan relasi antar data.
- Interaksi Database: Menangani seluruh query SQL seperti SELECT, INSERT, UPDATE, dan DELETE secara internal.
- Pada Laravel, Model dibalut secara elegan menggunakan teknologi Eloquent ORM.

Keuntungan Model Terisolasi

- Tidak ada kueri database (SQL) mentah yang bercampur di file tampilan HTML atau file logika Controller.
- Jika kita beralih DBMS (misal dari MySQL ke PostgreSQL), kita tidak perlu merombak seluruh kode aplikasi, cukup sesuaikan konfigurasi koneksi dan biarkan ORM/Model yang menerjemahkan kuerinya.
- Mempercepat manipulasi data karena baris database dibaca langsung sebagai Object berorientasi objek.

View: Presentasi & Rendering

Tanggung Jawab View

- Menyajikan data visual: Bertugas murni merender antarmuka pengguna (UI) berupa HTML, CSS, dan JavaScript.
- Penerima Pasif: Menerima data matang yang sudah diproses oleh Controller untuk ditampilkan.
- Bebas Logika Rumit: View tidak boleh melakukan kueri database langsung atau menjalankan kalkulasi bisnis kompleks.
- Di Laravel, View ditulis menggunakan file berekstensi `.blade.php`.

Blade Templating Engine

- Blade adalah templating engine bawaan Laravel yang sangat kuat namun ringan.
- Mendukung Pewarisan Layout (Layout Inheritance): Kita bisa mendefinisikan master template (header, footer, sidebar) sekali, lalu mewariskannya ke halaman lain.
- Menggunakan penulisan kontrol dinamis super bersih (seperti `@if`, `@foreach`, `@extends`) menggantikan sintaks PHP mentah yang berantakan.
- Otomatis menyaring teks variabel untuk mencegah celah keamanan XSS.

Controller: Jembatan Logika Bisnis

Tanggung Jawab Controller

- Pengendali Request: Menerima request HTTP dari Router yang berasal dari interaksi user (misal klik link atau submit form).
- Pengambil Data: Meminta data yang relevan dari Model sesuai instruksi request.
- Pemroses Logika: Menjalankan logika bisnis, validasi input formulir dasar, kalkulasi data, dsb.
- Pengirim Tampilan: Mengirimkan hasil olahan data ke View yang sesuai untuk ditampilkan.

Menjaga Kode Tetap Ramping

- Controller bertindak sebagai lalu lintas utama agar Model tidak menyentuh View, dan View tidak menyentuh Model.
- Menghindari tumpang tindih logika: Controller hanya mengoordinasikan interaksi.
- Semboyan Desain: 'Skinny Controller, Fat Model' - usahakan logika data diletakkan di dalam Model, sementara Controller tetap ramping hanya untuk mengarahkan alur.

Siklus Hidup Request (Request Lifecycle) MVC

1. Browser (User Action)

Pengguna membuka URL /mahasiswa atau mengirim form di browser.

2. Router

Menangkap request, mencocokkan URL, lalu memanggil controller yang tepat.

3. Controller

Menerima request, menganalisis parameter, lalu meminta data ke Model.

4. Model & Database

Model melakukan query, mengambil data dari DB, dan mengembalikan objek ke Controller.

5. View Renderer

Controller mengirimkan objek data ke View. Blade merender HTML final.

6. Response

HTML final dikirim kembali oleh Controller ke Browser pengguna untuk ditampilkan.

Manfaat Nyata Arsitektur MVC

1. Kolaborasi Efisien

- Pembagian Kerja Jelas: Tim desainer frontend dapat bekerja penuh di dalam folder View.
- Tim backend developer memprogram kueri dan algoritma di folder Model & Controller.
- Meminimalisir konflik merge pada Git secara drastis.

2. Reusabilitas Tinggi

- Model yang mendefinisikan kueri tabel Mahasiswa dapat dipanggil berkali-kali untuk berbagai kebutuhan.
- Dapat digunakan untuk halaman web HTML biasa.
- Dapat digunakan juga untuk endpoint JSON API (mobile app) tanpa menulis ulang kueri DB.

3. Mudah Ditest

- Unit testing otomatis dapat diimplementasikan dengan sangat mudah.
- Kita dapat menguji fungsionalitas logika Controller dan query database Model secara otomatis tanpa harus memuat tampilan UI secara manual.

BAGIAN 3: Ekosistem Framework Modern

Mengenal Peta Persaingan Framework Web Global: PHP vs Non-PHP

Perbandingan Framework PHP Terpopuler

Laravel (Fokus Kita)

- Sintaks sangat ekspresif, elegan, dan menyenangkan.
- Memiliki ekosistem tools terlengkap (Livewire, Herd, Sanctum, dll).
- Merupakan framework PHP paling dominan secara global saat ini.
- Komunitas raksasa dan dokumentasi yang luar biasa indah.

Symfony

- Sangat modular, fleksibel, kokoh, dan standar enterprise tinggi.
- Sering digunakan untuk aplikasi perbankan atau sistem korporasi skala besar.
- Komponen Symfony digunakan sebagai fondasi penyokong internal Laravel.

CodeIgniter 4 (CI4)

- Terkenal sangat ringan, jejak memori kecil, dan eksekusi cepat.
- Konfigurasi minimal, kurva belajar instan.
- Cocok untuk server dengan spek rendah.
- Namun, ekosistem bawaan dan modernisasinya masih berada di bawah Laravel.

Mengapa Laravel Begitu Istimewa?

Visi Taylor Otwell (2011)

- Taylor Otwell merancang Laravel untuk memulihkan kegembiraan menulis kode PHP (The PHP Framework for Web Artisans).
- Fokus pada sintaksis yang intuitif, bersih, dan ekspresif.
- Menyediakan solusi bawaan bagi seluruh fungsionalitas modern yang umumnya melelahkan untuk dibuat dari nol.

Ekosistem Terluas & Terpadu

- Laravel Herd & Sail: Mempermudah penyiapan lokal environment.
- Eloquent ORM: Manipulasi DB termudah di PHP.
- Laravel Sanctum & Passport: Proteksi API siap saji.
- Laravel Livewire: Pembuatan frontend interaktif reaktif tanpa Javascript berat.
- NativePHP: Mengemas web Laravel menjadi desktop application.

Wawasan Framework Non-PHP di Industri

Next.js (JS/TS)

- Berbasis React.js.
- Mendominasi dunia frontend web modern.
- Mendukung rendering server-side (SSR), static site generation (SSG), dan Server Actions full-stack.
- Sangat populer untuk web dinamis performa tinggi.

Django (Python)

- Semboyan: 'The web framework for perfectionists with deadlines'.
- Mengusung konsep 'batteries-included' (fitur bawaan super lengkap).
- Paling favorit untuk integrasi kecerdasan buatan (AI) dan pengolahan data masif karena kekuatan ekosistem Python.

Spring Boot (Java)

- Kokoh, berskala enterprise raksasa, keamanan tinggi.
- Menjadi pilihan dominan di sektor perbankan, fintech, dan korporat skala global.
- Sangat handal untuk menangani jutaan transaksi konkuren secara bersamaan.

Relevansi Laravel di Industri Modern

Kecepatan Pengembangan (TTM)

- Time-to-Market (TTM) yang cepat sangat krusial bagi startup.
- Dengan fitur boilerplating dan scaffolding instan Laravel, startup dapat merilis produk Minimum Viable Product (MVP) mereka ke pasar dalam hitungan minggu, bukan bulan.
- Memotong biaya riset dan infrastruktur kode secara drastis.

Skalabilitas & Permintaan Kerja

- Skala Enterprise: Dilengkapi driver bawaan untuk antrean (Queue) asinkron, sistem cache Redis, dan load balancer modern.
- Lowongan Kerja Melimpah: Industri lokal dan internasional sangat aktif mencari developer Laravel terampil.
- Ekosistem PHP saat ini masih menyokong lebih dari 75% website di internet global.

BAGIAN 4: Menyiapkan Lingkungan Pengembangan

Instalasi PHP 8.3+, Composer, Node.js, dan Struktur Proyek Laravel

Spesifikasi & Tools Minimum

Sebelum mulai mengoding Laravel, pastikan komputer Anda telah memenuhi spesifikasi kelayakan minimum berikut agar tidak terjadi crash atau konflik versi!

1. PHP 8.3+

Bahasa pemrograman inti. Laravel modern membutuhkan fitur penulisan tipe data yang presisi pada PHP versi terbaru.

2. Composer

Dependency manager PHP. Bertanggung jawab mengunduh framework Laravel serta paket-paket eksternal.

3. Node.js & NPM

Digunakan untuk memaketkan aset CSS & JS frontend secara kilat menggunakan bundler modern Vite.

4. Git & Editor

VS Code atau PhpStorm untuk editor kode, dan Git untuk kolaborasi kontrol versi tim.

Composer: Jantung Dependensi PHP

Mengapa Composer Penting?

- Manajemen Dependensi Terpusat: Composer mengelola seluruh pustaka (pustaka database, keamanan, log, mail, dll) agar tidak bentrok.
- Autoloading Otomatis: Composer menyediakan autoloading berstandar PSR-4, sehingga kita tidak perlu lagi menulis 'require' manual untuk setiap kelas PHP.
- Packagist Integration: Terhubung ke Packagist, perpustakaan global PHP yang berisi ratusan ribu paket siap pakai.

Dua File Kunci Composer

- composer.json:
File konfigurasi berbentuk JSON yang mencantumkan nama dan versi library yang secara eksplisit disini oleh proyek kita.
- composer.lock:
File pengunci yang mencatat versi persis library yang terpasang di lokal kita, menjamin seluruh anggota tim memiliki versi library yang identik.

Herd vs Sail: Pilih Environment Anda

Laravel Herd (Sangat Direkomendasikan)

- Ringan & Instan: Aplikasi GUI native untuk Mac & Windows.
- Tanpa Docker: Berjalan langsung di OS lokal tanpa beban virtualisasi.
- Mengintegrasikan PHP modern, Nginx web server, Composer, dan DNS lokal secara otomatis.
- Sempurna untuk mahasiswa karena menghindari beban RAM laptop yang berat akibat Docker.

Laravel Sail (Docker-based)

- Terisolasi Sempurna: Menyediakan environment yang terwadahi di dalam container Docker.
- Konsistensi Tim: Menjamin seluruh tim memiliki versi sistem operasi, DB, server, dan PHP yang sama presisi.
- Dijalankan dengan perintah terminal `./vendor/bin/sail up`.
- Membutuhkan RAM minimal 8GB-16GB untuk menjalankan Docker Daemon dengan lancar.

Membuat Proyek Laravel Baru

Langkah-Langkah CLI

1. Jalankan terminal / command prompt.
2. Masukkan perintah instalasi Composer.
3. Pindah masuk ke dalam folder proyek baru.
4. Jalankan server lokal Laravel.
5. Buka web browser kesayangan Anda dan akses alamat IP server lokal tersebut.
6. Amati halaman welcome statik bawaan Laravel yang berhasil dimuat.

```
# 1. Buat proyek via Composer
composer create-project laravel/laravel belajar-laravel
```

```
# 2. Masuk ke direktori proyek
cd belajar-laravel
```

```
# 3. Jalankan server lokal
php artisan serve
```

```
# Output di terminal:
# INFO Server running on [http://127.0.0.1:8000]
```

```
# 4. Akses URL tersebut di browser Anda!
```

Anatomi Struktur Folder Laravel

Direktori Utama

belajar-laravel/

|-- app/

|-- config/

|-- database/

|-- public/

|-- resources/

|-- routes/

+-- .env

Penjelasan Fungsi Folder

- app/: Rumah utama logika backend Anda. Di dalamnya terdapat Models/ (struktur data), Http/Controllers/ (pengendali alur), dan Middleware/.
- routes/: Folder definisi URL. File routes/web.php digunakan untuk mendefinisikan URL halaman web, routes/api.php untuk REST API JSON.
- resources/: Berisi folder views/ tempat meletakkan file template tampilan Blade (.blade.php), serta folder aset CSS/JS mentah.
- database/: Menyimpan migration (skema database), seeder (data dummy instan), dan factory.
- config/ & .env: Tempat konfigurasi global aplikasi dan environment variables seperti password database atau API keys.

Artisan CLI: Asisten Baris Perintah Anda

Mengapa Harus Artisan?

- Artisan adalah Command Line Interface (CLI) bawaan Laravel.
- Menghilangkan Kesalahan Manual: Membuat Controller, Model, and Migration secara manual rawan salah ketik nama class, folder, atau namespace.
- Artisan secara otomatis menulis file boilerplate dengan struktur kelas, namespace, dan pewarisan (extends) yang 100% presisi dan siap pakai.
- Sangat mempercepat produktivitas development web harian.

Cara Menjalankan Artisan

- Buka terminal di folder akar proyek Laravel Anda.
- Selaluawali dengan memanggil php artisan.
- Format: `php artisan [nama_command] [parameter]`
- Untuk melihat daftar seluruh perintah bawaan, jalankan:
`php artisan list`
- Untuk melihat bantuan pada perintah tertentu, panggil:
`php artisan help make:controller`

Cheat Sheet Perintah Artisan yang Sering Dipakai

php artisan serve

Menjalankan server lokal pengembangan untuk menguji web di browser.

php artisan make:controller [Nama]

Membuat file Controller baru di app/Http/Controllers/.

php artisan make:model [Nama] -m

Membuat file Model Eloquent sekaligus file Migrasi database (-m).

php artisan migrate

Mengeksekusi file migrasi untuk membangun/mengubah tabel DBMS MySQL.

php artisan route:list

Menampilkan tabel daftar seluruh rute/URL yang terdaftar di aplikasi.

php artisan make:middleware [Nama]

Membuat kelas filter request (middleware) baru di proyek Anda.

BAGIAN 5: Contoh Soal & Jawaban (Kodingan)

Latihan Soal Pemrograman Dasar Routing, Controller, dan View Laravel

Contoh Soal 1: Mendefinisikan Route GET

Soal 1: Routing Dasar

Pertanyaan:

Bagaimana cara mendefinisikan sebuah rute (route) baru bermetode GET pada file routes/web.php yang ketika pengguna mengakses alamat URL '/hello' di browser, server akan merespon dengan string teks: 'Selamat Belajar Laravel di ITS!'?

Gambarkan juga alur penulisan kodenya secara bersih!

Jawaban & Kode Penjelas

```
// routes/web.php
use Illuminate\Support\Facades\Route;

// Definisikan rute GET /hello
Route::get('/hello', function () {
    return 'Selamat Belajar Laravel di ITS!';
});
```

Contoh Soal 2: Route Parameter Opsional

Soal 2: Parameter dengan Default

Pertanyaan:

Buatlah sebuah rute GET di routes/web.php dengan pola URL '/user/{nama?}' di mana parameter '{nama}' bersifat opsional.

Jika parameter diisi di browser (misal /user/budi), tampilkan 'Halo, budi!'. Namun jika tidak diisi (misal /user), berikan nilai default 'Guest' dan tampilkan 'Halo, Guest!'.

Jawaban & Kode Penjelas

```
// routes/web.php
use Illuminate\Support\Facades\Route;

Route::get('/user/{nama?}', function ($nama = 'Guest') {
    // htmlspecialchars mencegah serangan XSS
    return 'Halo, ' . htmlspecialchars($nama) . '!';
});
```

Contoh Soal 3: Controller dari Artisan

Soal 3: Route ke Controller

Pertanyaan:

1. Apa perintah Artisan CLI untuk menghasilkan sebuah Controller bernama 'MahasiswaController'?
2. Di dalam controller tersebut, buatlah method public 'show(\$nrp)' yang mengembalikan teks NRP.
3. Tunjukkan cara mendaftarkan rute GET '/mahasiswa/{nrp}' di routes/web.php yang memanggil controller tersebut!

Jawaban & Kode Penjelas

```
# 1. Buat Controller lewat Artisan
php artisan make:controller MahasiswaController

// 2. app/Http/Controllers/MahasiswaController.php
class MahasiswaController extends Controller {
    public function show($nrp) {
        return 'Mahasiswa NRP: ' . htmlspecialchars($nrp);
    }
}

// 3. routes/web.php
use App\Http\Controllers\MahasiswaController;
Route::get('/mahasiswa/{nrp}', [MahasiswaController::class,
'show']);
```

Contoh Soal 4: Menghubungkan Controller ke View

Soal 4: Passing Data ke Blade View

Pertanyaan:

Buatlah sebuah controller 'AboutController' dengan method 'index()'. Method ini memuat file view bernama 'about.blade.php' dan melempar variabel 'nama' berisi string 'Teknik Informatika ITS'.

Tunjukkan juga bagaimana file 'about.blade.php' menangkap dan menampilkan data tersebut secara aman menggunakan kurung kurawal ganda (double curly braces) Blade!

Jawaban & Kode Penjelas

```
// 1. app/Http/Controllers/AboutController.php
public function index() {
    $nama = 'Teknik Informatika ITS';
    return view('about', compact('nama'));
}

<!-- 2. resources/views/about.blade.php -->
<!DOCTYPE html>
<html>
<body>
    <h1>Tentang Jurusan</h1>
    <!-- Double curly braces otomatis meng-escape teks -->
    <p>Selamat datang di {{ $nama }}!</p>
</body>
</html>
```

Contoh Soal 5: Membuat Route Resource

Soal 5: Struktur CRUD Otomatis

Pertanyaan:

Jika kita ingin mendesain sistem CRUD (Create, Read, Update, Delete) yang lengkap dan rapi untuk data 'Dosen':

1. Apa perintah Artisan CLI untuk membuat Controller yang sudah dibekali 7 method CRUD boilerplate?
2. Bagaimana cara menuliskan rute tunggal (single route) pada routes/web.php untuk langsung menangani seluruh kueri RESTful CRUD tersebut?

Jawaban & Kode Penjelas

```
# 1. Buat Resource Controller via Artisan
php artisan make:controller DosenController --resource

# Ini otomatis membuat method:
# index(), create(), store(), show(),
# edit(), update(), destroy()

// 2. Daftarkan di routes/web.php
use App\Http\Controllers\DosenController;

// Menangani 7 rute CRUD sekaligus
Route::resource('dosen', DosenController::class);
```

BAGIAN 6: Tugas Mandiri & Tantangan Mingguan

Penugasan Terstruktur Tatap Muka 1 Minggu untuk Dipresentasikan Minggu Depan

Tugas Mandiri: Laravel Sandbox Pertama

Tujuan Utama Tugas

- Memastikan setiap mahasiswa berhasil 100% dalam melakukan penyiapan (setup) lingkungan pengembangan Laravel lokal di komputernya.
- Melatih pemahaman mendalam tentang siklus navigasi dasar dari URL -> Router (web.php) -> Controller -> Blade View.
- Membiasakan penggunaan Git dan GitHub untuk publikasi kode program.
- Melatih softskill presentasi teknis & demo langsung.

Tema Proyek Sandbox

- Nama Tema: 'Sistem Informasi Statik Profil Mahasiswa (ITS Academic Profile)'.
- Sifat Tugas: Individu untuk instalasi koding, namun presentasi dirangkum berkelompok minggu depan.
- Deskripsi: Membangun aplikasi web minimalis yang menyajikan rincian data akademik pribadi, biodata diri, dan deskripsi rencana ide proyek Agentic AI yang ingin dikerjakan kelompok di akhir semester.

Spesifikasi Teknis Tugas (Wajib)

Daftar Fitur & Aturan Rute yang Wajib Diimplementasikan

1. Rute Beranda (GET '/')

- Memanggil method 'index' pada PageController.
- Merender view Blade 'home.blade.php' yang menyajikan tulisan selamat datang beserta Nama Lengkap & NRP Anda.

2. Rute Profil Jurusan (GET '/about')

- Memanggil method 'about' pada PageController.
- Merender view 'about.blade.php' berisi profil singkat Departemen Teknik Informatika ITS.

3. Rute Rencana Proyek Akhir (GET '/project-idea')

- Memanggil method 'project' pada PageController.
- Merender view 'project.blade.php' berisi penjelasan tentang sub-tema platform Agentic AI yang akan diangkat oleh kelompok Anda di akhir semester.

[PENTING] Catatan Arsitektur: Tidak boleh menggunakan Closure di routes/web.php untuk merender tampilan! Seluruh rute wajib didelegasikan ke PageController.

Tantangan Tambahan (Challenge!)

Tantangan 1: Kalkulator Dinamis

Untuk mendapatkan nilai bonus (A+), buatlah fitur kalkulator dinamis dengan aturan berikut:

- Buat rute GET '/hitung/{angka1}/{angka2}/{operasi}'
- Di dalam controller, lakukan kalkulasi sesuai dengan parameter 'operasi' yang dikirimkan (tambah, kurang, kali, bagi).
- Tampilkan hasil perhitungan di halaman web secara dinamis.
- Contoh: Mengakses /hitung/10/5/kali akan menampilkan text: 'Hasil dari 10 kali 5 adalah 50'.

Tantangan 2: Kustomisasi Visual

- Integrasi CSS: Untuk pertemuan pertama ini, integrasikan framework Bootstrap 5 atau Tailwind CSS menggunakan link CDN.
- Atur layout halaman agar terlihat rapi, memiliki navigasi menu (navbar) di bagian atas untuk berpindah antar halaman (Home, About, Project, Kalkulator).
- Pastikan visual web responsif saat diuji lewat handphone maupun laptop.

Kriteria Penilaian & Rubrik Evaluasi

Kesesuaian Teknis (40%)

- Seluruh rute wajib berjalan lancar tanpa error 404.
- Arsitektur rute ke Controller teratur rapi sesuai standar.
- Tidak ada kodingan yang hardcoded secara ilegal.

Kualitas Kode & UI (40%)

- Kerapian Kode (20%): Mematuhi PSR standar penulisan variabel, indentasi rapi, komentar secukupnya.
- Estetika UI & Respon (20%): Layout tertata rapi, link navigasi berfungsi, visual bersih dan profesional.

Bonus & Demo (20%)

- Nilai Plus Challenge (10%): Kalkulator dinamis berjalan tanpa cela.
- Penguasaan Konsep (10%): Kemampuan menjawab pertanyaan lisan saat sesi demo luring minggu depan.

Ketentuan Pengumpulan & Sesi Demo

Aturan Pengumpulan

- Git & GitHub:

Unggah folder tugas ke repositori GitHub masing-masing.

Pastikan folder vendor/ dan file .env TIDAK ikut terunggah (gunakan file .gitignore bawaan Laravel).

- Batas Waktu Pengumpulan:

Kirim tautan repositori GitHub dan screenshot halaman utama tugas melalui LMS ITS paling lambat H-1 (Pukul 23:59 WIB) sebelum kuliah Minggu 2 dimulai.

Mekanisme Demo Sesi Depan

- Sesi Presentasi Kelompok:

Di awal Pertemuan 2, setiap kelompok wajib menyiapkan slide presentasi minimal 5 slide yang merangkum hasil instalasi tim serta visualisasi alur rute yang berhasil dibangun.

- Pemilihan Acak:

Dosen akan menunjuk beberapa mahasiswa secara acak dari kelompok terpilih untuk mendemokan kodingan lokalnya.

Durasi demo maksimal 5 menit per mahasiswa.